Pranav H
(192525420)

SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA0720

COURSE OUTCOME COVERED

CO5: Measure network performance using key metrics with simulation tools. *(BL4,BL5)*

Assessment Tool Weightage: 30%

Annexure A

82/10

Pseudocode Writing Task (Algorithm Design)

1. In a network simulation using Cisco Packet Tracer, a student records total data received as 8000 bits in 4 seconds and observes that 200 packets were sent while 180 packets were received. Write a pseudocode algorithm to calculate the network throughput in bits per second and the packet loss percentage based on the given values. The algorithm should include validation to ensure that time and total packets sent are not zero to avoid errors. It should display both the throughput and packet loss results clearly. Add logic to classify the network as Efficient if throughput is greater than 1500 bps and packet loss is less than 10%, otherwise classify it as Inefficient. Finally, ensure the algorithm outputs a complete summary of the network performance based on the calculated metrics.
2. An organization has multiple network links connecting its branches. The administrator wants to monitor bandwidth utilization and identify overloaded links. Write a pseudocode algorithm that periodically collects the bandwidth usage of each network link and stores the values in an array or list. Use a loop to continuously monitor all links. Add conditions to detect when bandwidth utilization exceeds 80% and recommend switching traffic to a backup link. Explain how this algorithm supports efficient bandwidth management and prevents network congestion.
3. A company has six branch offices connected through multiple network links. Each link has different bandwidth, delay, and utilization values, which change throughout the day. The network administrator wants to continuously identify the best communication path between the headquarters and each

branch based on overall link quality rather than just the shortest distance.Write a pseudocode algorithm that periodically collects the bandwidth, delay, and utilization of every network link and stores the values in suitable arrays or lists. Use a loop to repeat the monitoring process at regular intervals. Calculate a link quality score for each path using the collected metrics and select the path with the highest score. Include conditions to detect when a link becomes overloaded or fails, automatically selecting an alternate path and generating an alert for the administrator. Explain how your algorithm improves network reliability, routing efficiency, and fault tolerance while adapting to changing network conditions.

RUBRICS FOR CALCULATION

	Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
A	Problem Understanding	20% 2	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
B	Algorithm Design	30% 3	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
C	Use of Control Structures	20% 2	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
D	Pseudocode Structure	10% 1	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
E	Completeness	20% 2	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

A1

	A	B	C	D	E	
Q1	2	2	1.5	1	2	8.5
Q2	2	2	1.5	1	2	8.5
Q3	1.5	2.5	2	.5	1.5	8
Q4	1.5	2	2	.5	1.5	7.5
Q5	2	2	1.5	1	2	8.5

SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO5: Measure network performance using key metrics with simulation tools. *(BL4,BL5)*

Assessment Tool Weightage: 30%

Annexure A

Pseudocode Writing Task (Algorithm Design)

1. In a network simulation using Cisco Packet Tracer, a student records total data received as 8000 bits in 4 seconds and observes that 200 packets were sent while 180 packets were received. Write a pseudocode algorithm to calculate the network throughput in bits per second and the packet loss percentage based on the given values. The algorithm should include validation to ensure that time and total packets sent are not zero to avoid errors. It should display both the throughput and packet loss results clearly. Add logic to classify the network as Efficient if throughput is greater than 1500 bps and packet loss is less than 10%, otherwise classify it as Inefficient. Finally, ensure the algorithm outputs a complete summary of the network performance based on the calculated metrics.
2. An organization has multiple network links connecting its branches. The administrator wants to monitor bandwidth utilization and identify overloaded links. Write a pseudocode algorithm that periodically collects the bandwidth usage of each network link and stores the values in an array or list. Use a loop to continuously monitor all links. Add conditions to detect when bandwidth utilization exceeds 80% and recommend switching traffic to a backup link. Explain how this algorithm supports efficient bandwidth management and prevents network congestion.
3. A company has six branch offices connected through multiple network links. Each link has different bandwidth, delay, and utilization values, which change throughout the day. The network administrator wants to continuously identify the best communication path between the headquarters and each branch based

on overall link quality rather than just the shortest distance.Write a pseudocode algorithm that periodically collects the bandwidth, delay, and utilization of every network link and stores the values in suitable arrays or lists. Use a loop to repeat the monitoring process at regular intervals. Calculate a link quality score for each path using the collected metrics and select the path with the highest score. Include conditions to detect when a link becomes overloaded or fails, automatically selecting an alternate path and generating an alert for the administrator. Explain how your algorithm improves network reliability, routing efficiency, and fault tolerance while adapting to changing network conditions.

RUBRICS FOR CALCULATION

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

ANSWERS

Pseudocode Writing Task (Algorithm Design) – Solutions to Questions 1, 2 & 3

Question 1: In a network simulation using Cisco Packet Tracer, a student records total data received as 8000 bits in 4 seconds and observes that 200 packets were sent while 180 packets were received. Write a pseudocode algorithm to calculate the network throughput in bits per second and the packet loss percentage based on the given values. The algorithm should include validation to ensure that time and total packets sent are not zero to avoid errors. It should display both the throughput and packet loss results clearly. Add logic to classify the network as Efficient if throughput is greater than 1500 bps and packet loss is less than 10%, otherwise classify it as Inefficient. Finally, ensure the algorithm outputs a complete summary of the network performance based on the calculated metrics.

Answer 1

1. Problem Understanding

Inputs: total_bits_received, time_seconds, packets_sent, packets_received (given values: 8000 bits, 4 seconds, 200 packets sent, 180 packets received).

Outputs: throughput (bps), packet_loss (%), and a network classification (Efficient / Inefficient) with a complete summary.

Constraints: time_seconds and packets_sent must not be zero (division-by-zero must be prevented); classification rule is throughput > 1500 bps AND packet_loss < 10%.

2. Pseudocode Algorithm

```
BEGIN
  INPUT total_bits, time_sec, packets_sent, packets_received

  // ---- Validation ----
  IF time_sec = 0 OR packets_sent = 0 THEN
     DISPLAY "Error: time and total packets sent must not be zero"
     STOP
  END IF

  // ---- Calculations ----
  throughput = total_bits / time_sec
  packet_loss = ((packets_sent - packets_received) / packets_sent) * 100

  // ---- Display results ----
  DISPLAY "Throughput = ", throughput, " bps"
  DISPLAY "Packet Loss = ", packet_loss, " %"

  // ---- Classification ----
  IF throughput > 1500 AND packet_loss < 10 THEN
     status = "Efficient"
  ELSE
     status = "Inefficient"
  END IF

  // ---- Summary ----
  DISPLAY "----- NETWORK PERFORMANCE SUMMARY -----"
  DISPLAY "Throughput: ", throughput, " bps"
  DISPLAY "Packet Loss: ", packet_loss, " %"
  DISPLAY "Network Status: ", status
END
```

3. Use of Control Structures

- Selection (IF...ELSE): used for zero-value validation and for the Efficient/Inefficient classification.
- Sequential structure: input, calculation, display, and summary steps run in a clear logical order.

- The algorithm is modular in spirit — validation, calculation, display, and classification are kept as distinct logical blocks, making it easy to convert into functions/modules in real code.

4. Worked Trace (with the given values)

throughput = 8000 / 4 = 2000 bps → greater than 1500 bps.

packet_loss = ((200 - 180) / 200) * 100 = 10% → NOT less than 10% (fails the strict '<' condition).

Since both conditions must be true, the network is classified as Inefficient, even though throughput alone looks good — this shows the importance of validating both metrics rather than just one.

5. Completeness

- Handles the zero-time / zero-packets edge case explicitly, preventing runtime errors.
- Handles both the calculation and the classification in one integrated flow.
- Produces a clearly labelled summary of every computed metric, satisfying the requirement for a complete performance report.

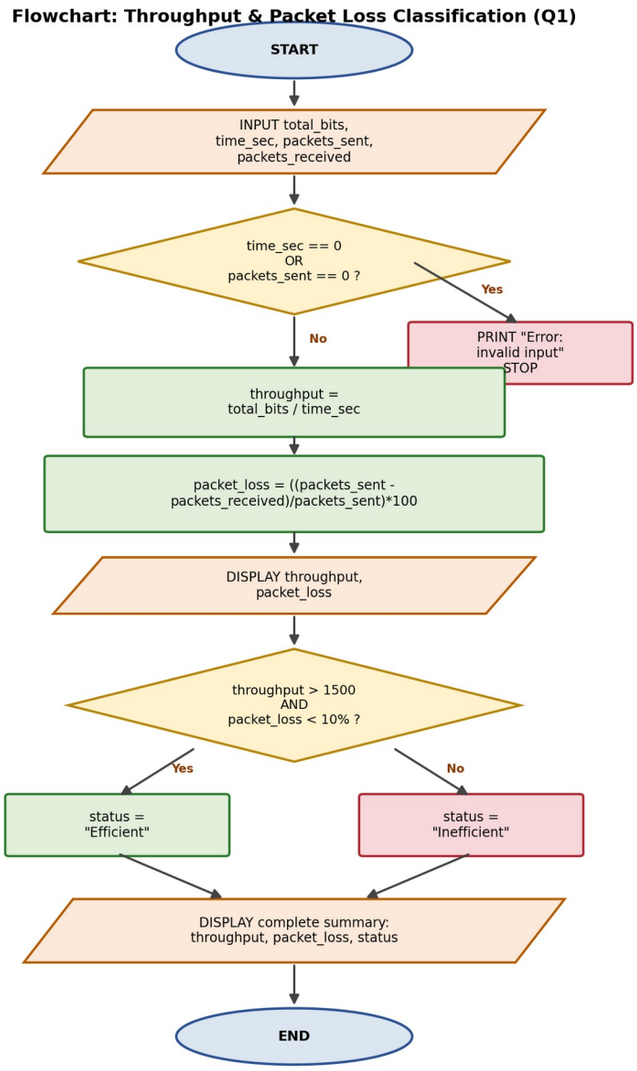

Figure 1: Flowchart for throughput calculation, packet loss, and network classification.

Question 2: An organization has multiple network links connecting its branches. The administrator wants to monitor bandwidth utilization and identify overloaded links. Write a pseudocode algorithm that periodically collects the bandwidth usage of each network link and stores the values in an array or list. Use a loop to continuously monitor all links. Add conditions to detect when bandwidth utilization exceeds 80% and recommend switching traffic to a backup link. Explain how this algorithm supports efficient bandwidth management and prevents network congestion.

Answer 2

1. Problem Understanding

Inputs: a list of network links (link_list) and, for each link, a corresponding backup link (backup_link_list); bandwidth usage is sampled repeatedly over time.

Outputs: a continuously updated usage_list, and real-time alerts/recommendations whenever a link crosses the 80% utilization threshold.

Constraints: monitoring must be continuous (not a one-time check) and must cover every link in the organization, storing results for later analysis.

2. Pseudocode Algorithm

```
BEGIN
  DECLARE link_list[1..N], backup_link_list[1..N]
  DECLARE usage_list[1..N]
  INITIALIZE link_list, backup_link_list  // configured by administrator

  monitoring_active = TRUE
  WHILE monitoring_active = TRUE DO
    FOR i = 1 TO N DO
      usage_list[i] = GET_BANDWIDTH_USAGE(link_list[i])

      IF usage_list[i] > 80 THEN
        DISPLAY "ALERT: ", link_list[i], " overloaded at ", usage_list[i], "%"
        DISPLAY "Recommendation: switch traffic to ", backup_link_list[i]
        SWITCH_TRAFFIC(link_list[i], backup_link_list[i])
      ELSE
        DISPLAY link_list[i], " is operating normally at ", usage_list[i], "%"
      END IF
    NEXT i

    WAIT(monitor_interval)   // e.g., pause 60 seconds before next cycle
  END WHILE
END
```

3. Use of Control Structures

- Outer WHILE loop: makes monitoring continuous/periodic rather than a single check, matching the requirement to "continuously monitor all links."
- Inner FOR loop: iterates over every link in the array so no link is skipped, regardless of how many branches exist (scales with N).
- IF...ELSE selection: distinguishes an overloaded link (>80%) from a normal one and triggers the appropriate action for each case.

4. Completeness

- Covers initial setup (link and backup arrays), continuous data collection, storage in usage_list, threshold-based decision-making, and a wait interval before repeating — a complete monitoring cycle.
- The design generalises to any number of links (N), so it remains correct if branches are added later.

5. How This Algorithm Supports Efficient Bandwidth Management

By storing usage values in an array/list every cycle, the administrator gets a continuously updated, historical view of bandwidth consumption across all links rather than a one-off snapshot, which supports trend analysis and capacity planning.

The 80% threshold acts as an early-warning trigger: traffic is automatically redirected to a backup link before a link becomes fully saturated (100%), which prevents the queuing delays, packet drops, and congestion collapse that occur once a link is overloaded. This proactive, threshold-based switching keeps the network responsive for all users and avoids a single busy link degrading the whole organization's connectivity.

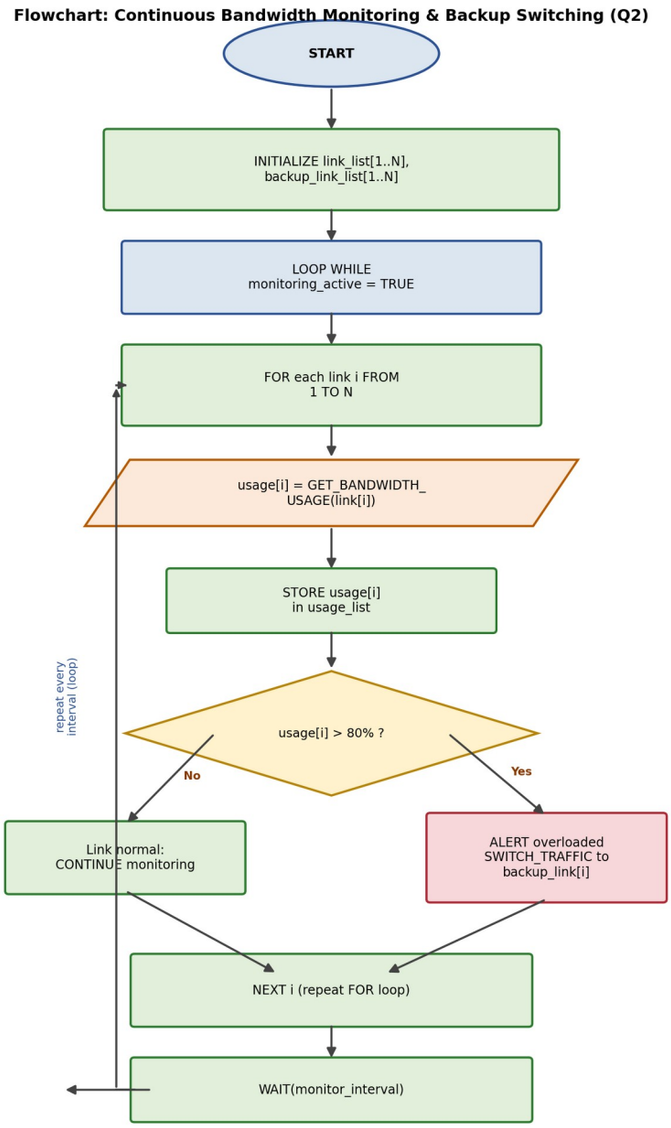

Figure 2: Flowchart for continuous bandwidth monitoring and automatic backup-link switching.

Question 3: A company has six branch offices connected through multiple network links. Each link has different bandwidth, delay, and utilization values, which change throughout the day. The network administrator wants to continuously identify the best communication path between the headquarters and each branch based on overall link quality rather than just the shortest distance. Write a pseudocode algorithm that periodically collects the bandwidth, delay, and utilization of every network link and stores the values in suitable arrays or lists. Use a loop to repeat the monitoring process at regular intervals. Calculate a link quality score for each path using the collected metrics and select the path with the highest score. Include conditions to detect when a link becomes overloaded or fails, automatically selecting an alternate path and generating an alert for the administrator. Explain how your algorithm improves network reliability, routing efficiency, and fault tolerance while adapting to changing network conditions.

Answer 3

1. Problem Understanding

Inputs: six branches, each connected via a link; for every link, three metrics are collected repeatedly: bandwidth, delay, and utilization.

Outputs: a per-link quality score, the best-scoring path to each branch, and automatic alerts/fail-over when a link is overloaded or down.

Constraints: metrics change throughout the day, so monitoring must repeat at regular intervals; path selection must use overall quality (a weighted combination of metrics), not just distance/hop count.

2. Pseudocode Algorithm

```
BEGIN
  DECLARE branches[1..6], links[1..6]
  DECLARE bandwidth[1..6], delay[1..6], utilization[1..6], score[1..6]
  SET w1 = 0.5, w2 = 0.3, w3 = 0.2   // weights for bandwidth, delay, utilization

  monitoring_active = TRUE
  WHILE monitoring_active = TRUE DO
    FOR i = 1 TO 6 DO
      bandwidth[i]    = GET_BANDWIDTH(links[i])
      delay[i]        = GET_DELAY(links[i])
      utilization[i]  = GET_UTILIZATION(links[i])

      // ---- Link quality score (higher = better) ----
      score[i] = (w1 * bandwidth[i]) - (w2 * delay[i]) - (w3 * utilization[i])

      // ---- Overload / failure detection ----
      IF utilization[i] > 90 OR STATUS(links[i]) = "DOWN" THEN
        DISPLAY "ALERT: link ", links[i], " to ", branches[i], " is overloaded/failed"
        alternate = SELECT_NEXT_BEST_LINK(score, exclude = i)
        SWITCH_PATH(branches[i], alternate)
        NOTIFY_ADMIN(links[i], alternate)
      END IF
    NEXT i

    // ---- Select best path per branch ----
    FOR i = 1 TO 6 DO
      best_index = INDEX_OF_MAXIMUM(score)
      DISPLAY "Best path to ", branches[i], " is ", links[best_index],
              " with score ", score[best_index]
    NEXT i

    WAIT(interval)   // repeat monitoring at regular intervals
  END WHILE
END
```

3. Use of Control Structures

- Outer WHILE loop: repeats the entire monitoring-and-scoring cycle at regular intervals, satisfying the requirement to adapt continuously to changing conditions.

- Inner FOR loops: iterate over all six links/branches for both metric collection/scoring and best-path selection, ensuring every branch is covered every cycle.
- IF selection: detects overload (>90% utilization) or failure (status = DOWN) and immediately triggers alternate-path selection and an administrator alert — without waiting for the next full cycle.
- Function/module calls (GET_BANDWIDTH, SELECT_NEXT_BEST_LINK, SWITCH_PATH, NOTIFY_ADMIN) keep the algorithm organised into clear, reusable responsibilities.

4. Completeness

- Handles metric collection, quality-score computation, best-path selection, overload/failure detection, automatic fail-over, and administrator alerting — covering every requirement in the question.
- The weighted scoring formula generalises the design: any link with higher bandwidth, lower delay, and lower utilization automatically scores higher, so best-path selection adapts correctly as conditions change throughout the day.

5. How This Algorithm Improves Reliability, Efficiency & Fault Tolerance

Reliability: continuously re-evaluating link quality (rather than checking once) means the chosen path is always based on current, not stale, network conditions.

Routing efficiency: using a composite score of bandwidth, delay, and utilization — instead of hop count or distance alone — ensures the path chosen is genuinely the fastest and least congested one available, not merely the geometrically shortest.

Fault tolerance: the overload/failure check triggers an automatic switch to the next-best-scoring alternate link and immediately alerts the administrator, so a single failed or congested link degrades service to only one branch briefly rather than causing a prolonged outage, and human intervention is informed in real time rather than after the fact.

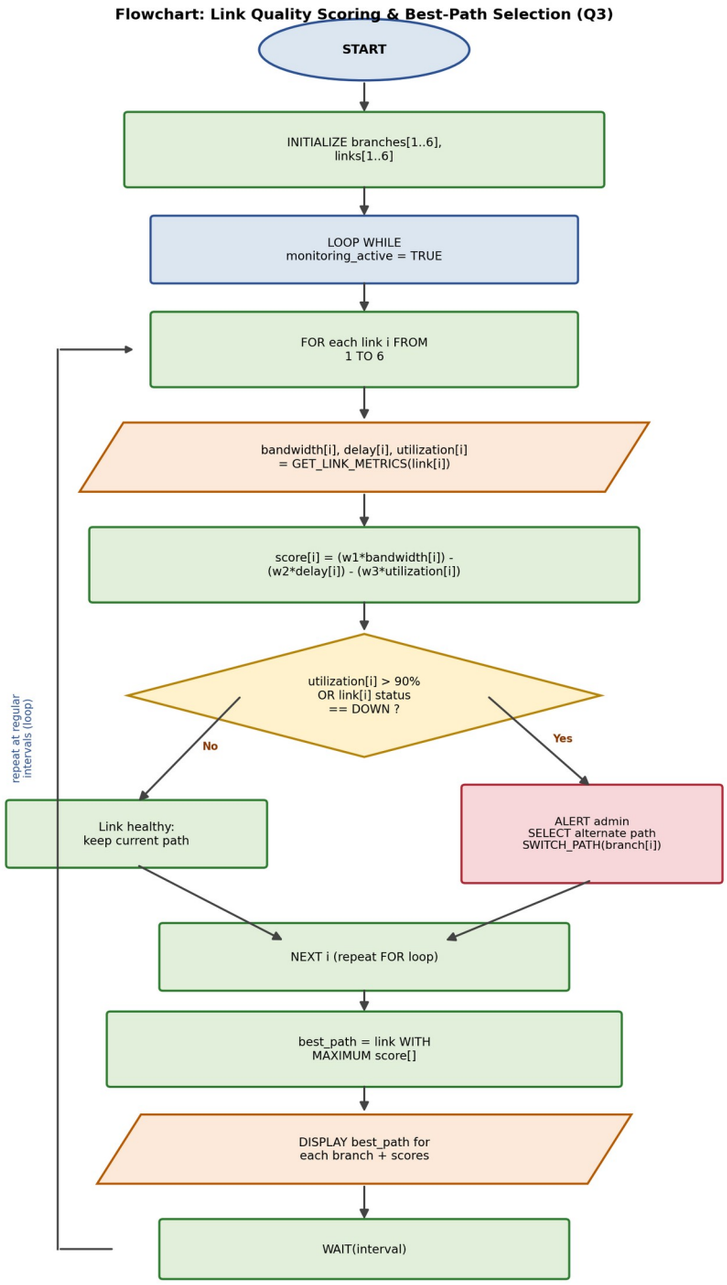

Figure 3: Flowchart for periodic link-quality scoring, best-path selection, and automatic fail-over.